

Student Management System in C++

Student Management System in C++

Objective

Develop a console-based Student Management System in C++ that efficiently manages student records using file handling and menu-driven operations.

Key Features

- Add new student records
- Update existing student details
- Delete student records
- Display all student information
- Search student by ID
- Persistent data storage using file handling
- User-friendly menu-driven interface

Expected Outcome

A functional C++ application that allows users to:

- Add student details
- Update records
- Delete records
- View stored information

The system stores data permanently using files, ensuring reliable record management even after the application is closed.

Technologies Used

- Programming Language: C++
- Concepts Used:
 - Classes & Objects
 - File Handling
 - Functions
 - Conditional Statements
 - Loops
 - Structures / OOP Concepts

Project Structure

StudentManagementSystem/

■

■■■ main.cpp

■■■ students.txt

■■■ README.md

Sample Student Record

| ID | Name | Age | Course | Marks |

|----|-----|----|-----|-----|

| 101 | Rahul | 20 | CSE | 85 |

ID	Name	Age	Course	Marks
101	Rahul	20	CSE	85

C++ Source Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <sstream>

using namespace std;

class Student {
public:
    int id;
    string name;
    int age;
    string course;
    float marks;

    void input() {
        cout << "\nEnter Student ID: ";
        cin >> id;

        cin.ignore();

        cout << "Enter Name: ";
        getline(cin, name);

        cout << "Enter Age: ";
        cin >> age;

        cin.ignore();

        cout << "Enter Course: ";
        getline(cin, course);

        cout << "Enter Marks: ";
        cin >> marks;
    }

    void display() {
        cout << "\nID: " << id;
        cout << "\nName: " << name;
        cout << "\nAge: " << age;
        cout << "\nCourse: " << course;
        cout << "\nMarks: " << marks << endl;
    }
};

vector<Student> students;

void saveToFile() {
    ofstream file("students.txt");

    for (auto s : students) {
        file << s.id << "|"
            << s.name << "|"
            << s.age << "|"
            << s.course << "|"
            << s.marks << endl;
    }
    file.close();
}
```

```

}

void loadFromFile() {
    students.clear();

    ifstream file("students.txt");

    string line;

    while (getline(file, line)) {
        Student s;

        stringstream ss(line);
        string temp;

        getline(ss, temp, '|');
        s.id = stoi(temp);

        getline(ss, s.name, '|');

        getline(ss, temp, '|');
        s.age = stoi(temp);

        getline(ss, s.course, '|');

        getline(ss, temp, '|');
        s.marks = stof(temp);

        students.push_back(s);
    }

    file.close();
}

void addStudent() {
    Student s;
    s.input();

    students.push_back(s);

    saveToFile();

    cout << "\nStudent Added Successfully!\n";
}

void displayStudents() {
    if (students.empty()) {
        cout << "\nNo Records Found!\n";
        return;
    }

    for (auto s : students) {
        s.display();
        cout << "-----";
    }
}

void searchStudent() {
    int id;
    cout << "\nEnter Student ID to Search: ";
    cin >> id;

    for (auto s : students) {
        if (s.id == id) {
            cout << "\nStudent Found:\n";
            s.display();
            return;
        }
    }

    cout << "\nStudent Not Found!\n";
}

void updateStudent() {
    int id;
    cout << "\nEnter Student ID to Update: ";
    cin >> id;

    for (auto &s : students) {
        if (s.id == id) {
            cin.ignore();

```

```

        cout << "Enter New Name: ";
        getline(cin, s.name);

        cout << "Enter New Age: ";
        cin >> s.age;

        cin.ignore();

        cout << "Enter New Course: ";
        getline(cin, s.course);

        cout << "Enter New Marks: ";
        cin >> s.marks;

        saveToFile();

        cout << "\nRecord Updated Successfully!\n";
        return;
    }
}

cout << "\nStudent Not Found!\n";
}

void deleteStudent() {
    int id;
    cout << "\nEnter Student ID to Delete: ";
    cin >> id;

    for (auto it = students.begin(); it != students.end(); ++it) {
        if (it->id == id) {
            students.erase(it);

            saveToFile();

            cout << "\nStudent Deleted Successfully!\n";
            return;
        }
    }

    cout << "\nStudent Not Found!\n";
}

int main() {
    loadFromFile();

    int choice;

    do {
        cout << "\n===== Student Management System =====\n";

        cout << "1. Add Student\n";
        cout << "2. Display Students\n";
        cout << "3. Search Student\n";
        cout << "4. Update Student\n";
        cout << "5. Delete Student\n";
        cout << "6. Exit\n";

        cout << "\nEnter Your Choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                addStudent();
                break;

            case 2:
                displayStudents();
                break;

            case 3:
                searchStudent();
                break;

            case 4:
                updateStudent();
                break;

            case 5:

```

```

        deleteStudent();
        break;

    case 6:
        cout << "\nExiting Program...\n";
        break;

    default:
        cout << "\nInvalid Choice!\n";
    }
} while (choice != 6);
return 0;
}

```

How to Compile and Run

Using GCC Compiler

Compile

g++ main.cpp -o student

Run

./student

Example Menu

===== Student Management System =====

1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Advantages of the Project

- Simple and beginner-friendly
- Demonstrates file handling in C++
- Helps understand CRUD operations
- Persistent data management
- Easy to extend with more features

Future Enhancements

- Login Authentication
- GUI Interface
- Database Integration (MySQL)
- Sorting and Filtering
- Export Reports as PDF
- Attendance Management

Conclusion

The Student Management System is a simple yet effective console-based project developed in C++. It demonstrates core programming concepts such as file handling, object-oriented programming, and menu-driven operations while providing persistent student record management functionality.